

■■ PentDash

Local Penetration Testing Dashboard
Architecture & Design Document

Version 1.0 · Linux (Ubuntu / Kali) · Localhost Deployment

1. Project Overview

PentDash is a locally hosted web dashboard that provides a clean, professional interface for running and organising penetration-testing tools. All tools are invoked as system commands via Python subprocess — no tool logic is reimplemented. The operator opens a single browser tab at **http://localhost:8000** and never needs a terminal during a session.

Core Principles

- All tools run natively on the OS — installed once via *install_tools.sh*.
- FastAPI backend manages tool execution, result storage, and the REST API.
- Plain HTML / JS / CSS frontend — no build step, opens directly in the browser.
- SQLite stores projects, targets, scans, findings and workflow templates.
- Findings are auto-tagged with severity: Critical / High / Medium / Low / Info.
- Parallel execution — all tools in a cluster fire simultaneously with one click.
- Sidebar navigation — each tool cluster is a section in the left nav.

2. Technology Stack

Layer	Choice	Purpose
OS	Linux — Ubuntu 22.04 / Kali	Native tool availability
Backend	Python 3.11 + FastAPI + Uvicorn	REST API, subprocess execution
Frontend	HTML 5 / Vanilla JS / CSS 3	No build step, fast load
Database	SQLite via SQLAlchemy ORM	File-based, zero config
Task runner	asyncio + ThreadPoolExecutor	Parallel tool execution
Communication	REST (JSON) — results on complete	Simple, no WebSocket complexity
Packaging	requirements.txt + install_tools.sh	Separate Python vs system deps

requirements.txt (Python packages)

```
fastapi uvicorn[standard] sqlalchemy python-dotenv aiofiles httpx
```

install_tools.sh (system tools — run once)

```
sudo apt update && sudo apt install -y \ nmap masscan recon-ng theharvester nikto wpscan \
sqlmap hydra medusa ncrack john hashcat aircrack-ng \ tshark tcpdump ffuf gobuster dirsearch
wfuzz \ metasploit-framework exploitdb impacket-scripts # Go-based tools go install
github.com/owasp-amass/amass/v4/...@latest go install
github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest go install
github.com/tomnomnom/assetfinder@latest go install
github.com/projectdiscovery/naabu/v2/cmd/naabu@latest go install
github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest go install
github.com/feroxbuster/feroxbuster@latest # Python-based tools pip install shodan droopescan
sstimap xsstrike commix \ crackmapexec bloodhound trufflehog gitleaks \ semgrep trivy checkov
pentestgpt --break-system-packages
```

3. UI Layout & Navigation

The interface is a single-page application divided into three zones:

Zone	Width	Content
Top Bar	100%	App name · Active project name · Project switcher button
Left Sidebar	220 px	Vertical list of tool clusters — click to load that cluster's panel
Main Panel	flex 1	Cluster heading · Shared input fields · Tool list with Run All button · Results area

Sidebar Cluster Order (customisable later)

The order below is the default. Each entry becomes one sidebar item.

#	Sidebar Label	Tools inside
1	Recon & Info Gathering	Nmap, Masscan, Amass, Subfinder, Assetfinder, Naabu, Recon-ng, theHarvester, Shodan, Cloudfox
2	Directory & Brute-Force	FFUF, Gobuster, Dirsearch, Feroxbuster, Wfuzz
3	Vulnerability Scanning	Nuclei, Nikto, W3af, Nessus CLI, WPScan, Droopescan
4	Web App Exploitation	SQLmap, SSTImap, Commix, XSSStrike, Ggau
5	Exploit Frameworks & Post-Exploit	Msfconsole, Searchsploit, Impacket, CrackMapExec / NetExec, BloodHound.py, Mimikatz
6	Password & Credential	Hashcat, John the Ripper, Hydra, Medusa, Ncrack, Crowbar
7	Wireless & Sniffing	Aircrack-ng, Tshark, Tcpdump, Bettercap
8	Code & Secret Analysis	Semgrep, TruffleHog, Gitleaks, Trivy, Checkov
9	AI Attack Agents	PentestGPT, PentAGI, Deadend CLI, Pentest-Swarm-AI

4. Shared Input Fields per Cluster

Each cluster panel shows a set of shared fields at the top. When the operator clicks **Run All**, every tool in the cluster receives these values automatically — no need to re-enter them per tool. Fields marked * are required before the button activates.

Cluster	Field	Type	Example / Notes
Recon & Info Gathering	Target IP / CIDR *	text	192.168.1.0/24
	Target Domain *	text	example.com
	Shodan API Key	password	sk-... (stored in .env)
	Scan Speed	select	T1 Sneaky / T3 Normal / T5 Insane
	Output Format	select	Normal / XML / Grepable
Directory & Brute-Force	Target URL *	text	https://example.com
	Wordlist Path *	text	/usr/share/wordlists/dirb/common.txt
	Extensions	text	php,html,js (comma-separated)
	Threads	number	50
	Follow Redirects	toggle	On / Off
Vulnerability Scanning	Target URL / IP *	text	https://example.com or 10.0.0.1
	CMS Type	select	Auto-detect / WordPress / Drupal / Joomla
	Auth Credentials	text	user:pass (for authenticated scans)
	Severity Filter	select	All / Critical / High / Medium
Web App Exploitation	Target URL *	text	https://example.com/page?id=1
	Parameter	text	id (for injection testing)
	Cookie / Session	text	PHPSESSID=abc123
	Proxy	text	http://127.0.0.1:8080 (optional)
Exploit Frameworks	Target IP *	text	192.168.1.50
	Target Port	number	445
	Domain	text	corp.local
	Username	text	administrator
	Password / Hash	text	NTLM hash or plaintext
Password & Credential	Hash File / Target *	text	/path/to/hashes.txt or IP
	Wordlist *	text	/usr/share/wordlists/rockyou.txt
	Hash Type	select	MD5 / SHA1 / NTLM / bcrypt / Auto
	Service (online)	select	SSH / FTP / HTTP / SMB / RDP
	Threads	number	4
Wireless & Sniffing	Interface *	text	wlan0 / eth0
	BSSID / Target MAC	text	AA:BB:CC:DD:EE:FF

	Channel	number	6
	Capture Duration (s)	number	60
Code & Secret Analysis	Repo / Path *	text	/home/user/project or git URL
	Branch	text	main
	Ruleset	select	Default / OWASP / CIS / Custom
AI Attack Agents	Target *	text	https://example.com or 10.0.0.1
	Objective	textarea	Identify SQL injection vulnerabilities
	AI Model / API Key	text	gpt-4o + sk-...

5. Workflow & Tool Chaining

A **Workflow** is an ordered list of clusters (or individual tools) that run sequentially, where the output of one step can be fed as input to the next. Within each step all selected tools run in **parallel**.

How a Workflow Runs

- Step 1 — Recon cluster fires all tools in parallel. Results stored in DB.
- Step 2 — Dir Brute-Force cluster picks up discovered URLs from Step 1 automatically.
- Step 3 — Vuln Scan uses the live hosts found in Step 1 as its target list.
- Each step waits for all parallel tools to complete before the next step starts.

Workflow Templates

Templates are saved workflows with a name and description. Pre-built templates included:

Template Name	Steps included	Typical Use Case
Full Web Recon	Recon → Dir Brute-Force → Vuln Scan	External web application
Internal AD Sweep	Recon → Exploit Frameworks → Password	Internal Active Directory
Bug Bounty Quick	Recon → Web App Exploitation → Vuln Scan	Bug bounty scope
DevSecOps Check	Code & Secret Analysis	CI/CD pipeline review
Custom	User-defined — drag & drop clusters into desired order	Any engagement

6. Data Model (SQLite)

Table	Key Columns	Purpose
projects	id, name, client, target_scope, created_at	Top-level engagement container
targets	id, project_id, value, type (ip/domain/url)	Individual targets per project
scans	id, project_id, cluster, tool, command, status, started_at, finished_at	One row per tool execution
results	id, scan_id, raw_output, parsed_json	Raw + parsed output
findings	id, scan_id, title, severity, description, evidence	Auto-extracted findings
workflows	id, name, steps_json, created_at	Saved workflow templates

Severity Tagging Logic

After each tool completes, a lightweight parser inspects the raw output for known patterns (CVE identifiers, CVSS scores, keyword lists per tool) and assigns one of:

Severity	Colour	Examples of trigger patterns
Critical	Red	CVSS ≥ 9.0 · RCE · command injection confirmed · auth bypass
High	Orange	CVSS 7–8.9 · SQLi confirmed · open admin panel · exposed secrets
Medium	Yellow	CVSS 4–6.9 · self-XSS · outdated software version · weak cipher
Low	Blue	CVSS 1–3.9 · info disclosure · non-sensitive header missing
Info	Grey	Open ports · DNS records · banner grabs · directory listing

7. Backend API Endpoints

Method	Path	Description
GET	/api/projects	List all projects
POST	/api/projects	Create new project
GET	/api/projects/{id}/scans	All scans for a project
POST	/api/run/cluster	Run all tools in a cluster (parallel)
POST	/api/run/workflow	Execute a saved workflow
GET	/api/scans/{id}/results	Fetch results for a scan
GET	/api/findings?project_id=&severity=	Filtered findings
GET	/api/workflows	List saved workflow templates
POST	/api/workflows	Save a new workflow template
DELETE	/api/workflows/{id}	Delete a workflow template
GET	/health	Backend health check

8. Project File Structure

```
pentdash/ ■■■■ backend/ ■ ■■■■ main.py # FastAPI app, router registration ■ ■■■■ database.py #
SQLAlchemy engine + session ■ ■■■■ models.py # ORM table definitions ■ ■■■■ schemas.py #
Pydantic request/response models ■ ■■■■ executor.py # subprocess runner + parallel logic ■ ■■■■
parser.py # Output parsers + severity tagger ■ ■■■■ routers/ ■ ■ ■■■■ projects.py ■ ■ ■■■■
scans.py ■ ■ ■■■■ findings.py ■ ■ ■■■■ workflows.py ■ ■■■■ tools/ ■ ■■■■ recon.py # Command
builders for cluster 1 ■ ■■■■ dirbrute.py # Command builders for cluster 2 ■ ■■■■ vulnscan.py
■ ■■■■ webapp.py ■ ■■■■ exploit.py ■ ■■■■ password.py ■ ■■■■ wireless.py ■ ■■■■ sast.py ■ ■■■■
ai_agents.py ■■■■ frontend/ ■ ■■■■ index.html # Single HTML entry point ■ ■■■■ css/ ■ ■ ■■■■
style.css ■ ■■■■ js/ ■ ■■■■ app.js # Router + state ■ ■■■■ api.js # Fetch wrappers for REST API
■ ■■■■ clusters.js # Cluster panel renderer ■ ■■■■ workflows.js # Workflow builder UI ■■■■
data/ ■ ■■■■ pentdash.db # SQLite database (auto-created) ■■■■ requirements.txt ■■■■
install_tools.sh ■■■■ start.sh # uvicorn main:app --reload ■■■■ .env # API keys (Shodan, etc.)
```

9. How to Start the Application

First-time setup

```
git clone https://github.com/you/pentdash cd pentdash pip install -r requirements.txt
--break-system-packages bash install_tools.sh cp .env.example .env # add Shodan key etc.
```

Start the backend

```
cd backend uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

Open the UI

```
# Open in browser http://localhost:8000
```

The backend serves the *frontend/* folder as static files, so no separate web server is needed. One command starts everything.